

ЗАЩИТА ОБЪЕКТОВ ОПЕРАЦИОННЫХ СИСТЕМ

В общем случае включает 3А=Аутентификация (идентифицирует субъекта, попадающего в систему=КТО), Авторизация (определяет для субъекта разрешенные действия в системе=ЧТО МОЖНО ДЕЛАТЬ), Аудит (фиксация в системном журнале событий, связанных с доступом к защищаемым ресурсам системы=ЧТО СДЕЛАНО).

Защита ОС – механизмы защиты объектов системы от их окружения. Механизмы различны для различных частей ОС. Вся информация в ВС расположена на различных устройствах памяти. Задача защиты – организовать защиту информации разделив пространство адресов этих устройств на области и разрешить только контролируемый обмен информацией между областями. Каждая область должна быть защищена от незаконных попыток получить / изменить информацию со стороны внешнего окружения. Здесь возникает 2 проблемы: эффективность охраны, эффективность доступа. Области адресуемого пространства должны быть ограждены стенами с воротами для обеспечения коммуникационных связей. Стены должны быть связаны с механизмом адресации, адресные ссылки внутри области не должны требовать обширных проверок, адресные ссылки на объекты вне данной области должны реализовываться через механизм – ворота. Коммуникационные связи (ворота) должны охраняться механизмом контроля, который разрешал бы доступ только субъектам, имеющим на это право.

Способы возведения стен: **права доступа, обеспечение секретности пути к объекту, применение методов кодирования.**

Права доступа. При сегментированном пространстве адресов каждый адрес внутри сегмента проверяется на принадлежность диапазону от начального адреса до начального адреса + длина сегмента. Т. о. адресный механизм возводит **Стену** вокруг всех адресов внутри сегмента и **Ворота**. Сами сегменты защищаются через таблицу сегментов при проверке законности обращения (в соответствии со статусом защиты) со стороны процесса, выдавшего ссылку.

Аналогична защита файлов – при попытке процесса получить доступ к файлу, его запрос проходит через СУФ, которая проверяет права процесса на соответствие статусу защиты. Не должно быть способов получения доступа к файлу минуя СУФ=ворота.

Другой способ защиты – **обеспечение секретности пути к объекту**. Объект не защищен, но его местоположение известно только определенным привилегированным процессам (проводникам)=ворота. Этот механизм не является абсолютно непроницаемым. Пользователь может по ошибке или намеренно получить доступ к объекту и повредить его.

Методы кодирования. Информация расположена в незащищенных областях, но в закодированном виде. Для раскодирования процесс должен знать ключ (известный только привилегированным процессам) к коду. Ворота=код, ключ к нему, привилегированные процессы.

При любом способе организации защиты объектов ОС наиболее предпочтительным механизмом являются **мониторы защиты**, выполняющие роль ворот. Мониторы, обеспечивающие синхронизацию, могут быть расширены для обеспечения защиты. Мониторы могут иметь программно-техническую реализацию.

СУФ – программный монитор.

Механизм логической адресации – имеет аппаратную поддержку.

Каждый объект может иметь собственный монитор защиты. Но с точки зрения эффективности целесообразно для похожих объектов иметь общие мониторы. Решение: разбиение управления защитой по группам объектов. Каждый монитор защиты служит воротами для большого класса сходных объектов (например, СУФ – требует соблюдения правил доступа для класса объектов – файлы).

Домены и возможности

В вопросах защиты ОС различают:

- активные субъекты (обычно процесс);
- пассивные объекты (обычно фрагменты памяти, файлы).

Субъект и объект с течением времени могут меняться ролями. Часто различные субъекты получают одинаковые права в отношении различных объектов. Полезно выделить классы прав доступа в отдельные категории – домены. Домен определяет контекст защиты, по которому может действовать некоторый субъект (права доступа). Процесс меняет свой статус защиты, меняя домен в котором он работает. Разделение статуса защиты на домены позволяет обращаться с правами доступа независимо от субъектов (это гибкое средство для совместного использования механизмов защиты).

Объекты, субъекты и домены – именуются уникальными идентификаторами. Это дает возможность монитору защиты контролировать законность прав доступа. Конкретное право доступа некоторого субъекта в отношении некоторого объекта – возможность (например, доступ только для чтения в отношении некоторого файла). Одна из проблем защиты – изменение прав доступа. Решение проблемы: процесс, раздающий права, имеет возможность их отбирать.

Описание статуса защиты

В произвольный момент времени имеется:

Множество доменов $X=\{x(1),x(2),\dots,x(n)\}$.

Множество объектов $Y=\{y(1),y(2),\dots,y(m)\}$.

Статус защиты описывается функциональным отображением $F: X*Y \rightarrow A$, множество $A=\{A1,...,AN\}$ – множество атрибутов доступа (только чтение, чтение и запись, только выполнение, ...). Домены рассматриваются независимо от субъектов, в любой момент времени существует способ связать заданного субъекта с определенным доменом. Статус защиты не статичен! Домены и объекты непрерывно создаются, уничтожаются, изменяются. Субъекты меняют домены, в которых они действуют.

Если отображение F зависит только от значений $x(i)$ и $y(j)$, то статус защиты хорошо описывается **матрицей**, строки которой задают имена доменов, а столбцы – имена объектов в системе. Для каждого домена и каждого объекта матрица определяет права данного домена на данный объект (в виде атрибутов доступа). **Обычно домен имеет права по отношению к небольшому числу объектов системы, тогда матрица содержит большое число пустых позиций.** Матричная модель может быть обобщена до функциональной, если в качестве элементов матрицы использовать процедуры (функции от др. переменных), проверяющие законность доступа к объекту.

В матричной схеме защиты должны запрещаться бесконтрольные изменения элементов матрицы (в противном случае не будет защиты). Но права доступа должны изменяться и для защиты изменений нужны специальные меры. Защиту изменений можно реализовать обращаясь с доменами как с объектами. Матрицу необходимо расширить, включив в нее столбцы, соответствующие доменам и изменять их.

Для реализации монитором защищенных изменений каждому элементу матрицы (каждому праву доступа) приписываются дополнительные специальные атрибуты: **владелец, управление, признак копирования**.

Атрибут **управление** какого-либо домена $x(i)$ над другим доменом $x(j)$ означает, что домен $x(i)$ полностью контролирует домен $x(j)$. Возможна иерархия доменов.

Атрибут **владелец** домена $x(i)$ по отношению к объекту y означает, что объект y принадлежит домену $x(i)$.

Атрибут **признак копирования** разрешает копирование права доступа в другой домен.

Статус защиты системы (сужение и расширение прав доступа к объектам системы в различных доменах) управляется правилами изменения элементов матрицы.

Правила расширения прав.

Правило 1. Домен $x(i)$ может передавать атрибуты доступа элементу $A(x(j),y)$, устанавливая или нет **признак копирования**, если домен $x(i)$ имеет атрибут **владелец** по отношению к объекту y .

Правило 2. Домен $x(i)$ может скопировать атрибут доступа A_k в элемент $A(x(j),y)$, если атрибут A_k находится в соответствующем элементе $A(x(i),y)$ и для него установлен атрибут **признак копирования**.

Правила сужения прав.

Правило 3. Домен $x(i)$ может удалить атрибуты доступа из элемента $A(x(j),y)$, если он имеет атрибут **управление** над доменом $x(j)$.

Правило 4. Домен $x(i)$ может удалить атрибуты доступа из элемента $A(x(j),y)$, если домен $x(i)$ имеет атрибут **владелец** по отношению к объекту y и, кроме того, домен $x(j)$ не имеет защищенного доступа к объекту y . Домен $x(j)$ принимает предохранительные меры против отмены его прав, отказываясь признать какие либо права доступа, которые не защищены.

Данную схему и набор правил для практической реализации необходимо дополнить:

1. Связями субъектов с доменами.
2. Перемещениями субъектов от одного домена к другому.

3. Возможностью создания новых доменов.
4. Возможностью удаления ненужных объектов или доменов из описания статусов защиты.

Схемы реализации защиты

Задача – найти компромисс между временными накладными расходами (определяются организацией структуры данных, описывающей схему защиты), издержками памяти, гибкостью системы защиты.

1. Для представления статуса защиты используется **матрица**. Тогда в таблице **T** описание статуса защиты состоит из триплетов [**домен, объект, атрибуты доступа**]. Поиск элемента **A(x,y)** ведется по глобальной сильно разреженной таблице **T**.

Недостатки схемы:

- а) Таблица **T** очень велика и вся не умещается в памяти (возможно хранение только отдельных фрагментов **T**).
- б) Трудно осуществлять объединение объектов или доменов в группы.
- в) сложно реализовать поиск объектов, к которым некоторый домен имеет доступ или по отношению, к которым он является владельцем.

2. Для представления статуса защиты используется **список возможностей** (строка матрицы). Все объекты, к которым имеет доступ некоторый домен включают в список, относящийся к данному домену – [**y,A(x,y)**] – это возможности, соответствующие правам доступа домена **x** к объекту **y**. Такие списки можно хранить в области памяти, защищенной ключом с доступом через высоко привилегированный монитор защиты. Удобное решение для пользователей ОС.

3. Для представления статуса защиты используется **список управления доступом** (столбец матрицы). Все домены, имеющие доступ к конкретному объекту, объединяются в список – [**x,A(x,y)**] – элементы списка задают атрибуты доступа для каждого домена, имеющего права доступа к объекту **y**. Для каждого объекта создавший его субъект задает процедуру и список управления доступом. Процедура отвечает за проверку доменов, которые пытаются получить доступ к данному объекту. Удобное решение для администратора ОС.

4. Для представления статуса защиты используется **механизм замок-ключ** с уникальными битовыми наборами. В каждом домене имеется список объектов с битовым набором для каждого объекта, битовый набор (ключ) служит для идентификации прав доступа к данному объекту. С каждым объектом связывается список уникальных битовых наборов (замков) для

каждого из которых указаны права доступа. Для получения доступа к объекту субъект предъявляет ключ монитору защиты, охраняющему данный объект. Монитор устанавливает соответствие ключа и замка, связанного с данным объектом и разрешает/запрещает доступ в зависимости от требуемого вида доступа. Схема **замок-ключ** эффективна и гибка:

- а) с битовыми наборами, соответствующими возможностям можно действовать независимо от значений наборов;
- б) отмена и изменение прав доступа осуществляется изменением списка битовых наборов (замков), связанного с данным объектом.

Недостатком является необходимость поддержки длинных битовых наборов (для исключения повторения битовых наборов).

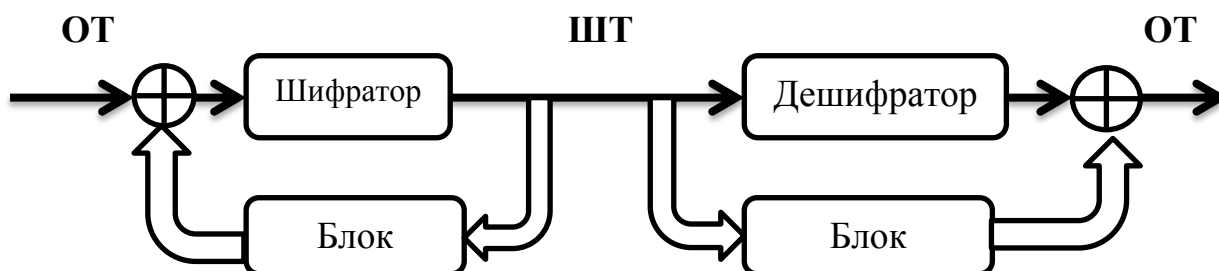
Схемы 2-4 более выигрышны по быстродействию, т.к. проверки законности доступа ими ограничиваются более локализованным поиском.

Криптосистема

Криптосистема выполняет 2 преобразования – шифрование (Ш) и дешифрование (Д).

Критерий качества криптосистемы: сохранение свойств секретности при общеизвестной сущности алгоритмов Ш и Д. Это достигается с помощью ключа К. Различают симметричные (с одним ключом, DES) и несимметричные (с двумя ключами – открытым и секретным, RSA) алгоритмы шифрования. Применяется обычно блочное кодирование.

В **симметричных** алгоритмах шифрования (отличающихся относительно невысокой трудоемкостью) существует опасность, что повторяющиеся группы данных открытого текста (ОТ) будут явно выражены в зашифрованном тексте (ШТ). Тогда по зашифрованному тексту и известным фрагментам открытого текста возможно восстановление ключа. Решением данной проблемы является метод сцепления или режим шифрования с обратной связью. Суть данного метода состоит в том, чтобы включить в первый кодируемый блок случайное число, а далее шифровать блоки в зависимости от значений предыдущих блоков.



Шифрование:

Перед шифрованием каждого блока ОТ к нему прибавляется по модулю 2 (исключающее или) содержание предыдущего зашифрованного блока (**по обратной связи**). В результате очередной зашифрованный блок зависит от содержимого всего предыдущего ОТ.

$Ш_i = E(O_i + Ш_{i-1})$, где O_i – i-й блок открытого текста, $Ш_i$ – i-й блок зашифрованного текста, E – функция шифрования.

Дешифрование:

$$O_i = E^{-1}(Ш_i) + Ш_{i-1}.$$

При дешифровании дешифруется очередной зашифрованный блок и складывается по модулю 2 с предыдущим зашифрованным блоком (**по прямой связи**).

Т.о. любое изменение ОТ многократно повторяется в ШТ!

НО ошибки, возникающие при передаче ШТ по линиям связи – локализуются только в 2-х блоках: 1) в том блоке, где произошла ошибка ($i-m$), 2) в следующем за ним блоке ($i+1-m$).

Несимметричные алгоритмы шифрования основаны на необратимых/односторонних (трудно обратимых, обратимых с очень высокой трудоемкостью) функциях. Примером несимметричного крипто алгоритма является метод RSA (Rivest, Shamir, Adleman), основанный на необратимости функции возведения в степень по модулю большого числа. Суть метода состоит в следующем:

1. Случайно выбираются 2 очень больших ПРОСТЫХ числа p и q .
2. Вычисляются 2 произведения $n=p*q$ и $m=(p-1)*(q-1)$.
3. Выбирается случайное целое число E , не имеющее общих сомножителей с m .
4. Находится целое число D , такое, что $D*E=1$ по модулю m .
5. Исходный открытый текст ОТ разбивается на блоки X длины менее n : $0 < X < n$.
6. Для шифрования блока сообщения необходимо вычислить $c = x^E$ по модулю n .
7. Для дешифрования вычисляется $x = c^D$ по модулю n .

Таким образом для шифрования сообщения необходимо знать пару чисел (E,n) , а чтобы дешифровать – пару чисел (D,n) . При шифровании первая пара – это открытый ключ, а вторая – закрытый.

Зная открытый ключ (E,n) , можно вычислить значение закрытого ключа D . Необходимым промежуточным действием в этом преобразовании является нахождение чисел p и q , для чего нужно разложить на простые множители очень большое число n , на что требуется очень много времени. Огромная вычислительная сложность разложения большого числа на простые множители обеспечивает высокую криптостойкость алгоритма RSA.

Программная реализация крипто алгоритмов типа RSA существенно сложнее и значительно менее производительна, чем реализация классических крипто алгоритмов типа DES. Обусловлено это сложностью реализации модульной

арифметики (с высокой разрядностью обрабатываемых чисел). Обычно RSA используют для:

1. Шифрования небольших объемов информации;
2. Рассылки классических секретных ключей крипто алгоритмов типа DES;
3. Реализации алгоритмов цифровой подписи.

Сравнительные характеристики алгоритмов шифрования

Характеристика	DES	RSA
Скорость шифрования	Высокая	Низкая
Используемая функция шифрования	Перестановка и подстановка	Возведение в степень
Длина ключа	56 бит	Более 500 бит
Наименее затратный крипто анализ (его сложность определяет стойкость алгоритма)	Перебор по всему ключевому пространству	Разложение числа на простые множители
Время генерации ключа	Миллисекунды	Минуты
Тип ключа	Симметричный	Асимметричный